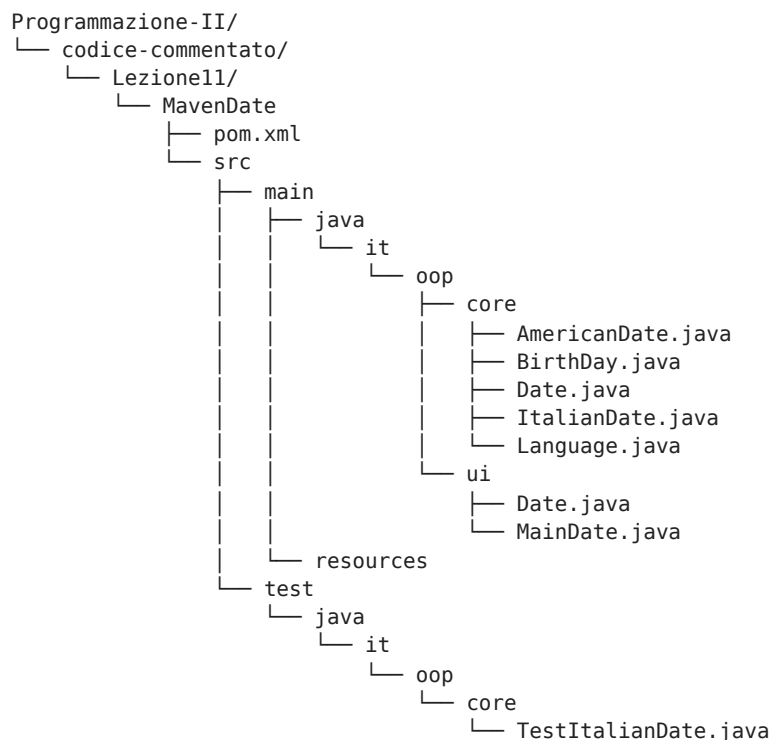


# Lezione 11: Polimorfismo, Specializzazione delle Sottoclassi e Metodo Equals

Questo notebook raccoglie tutto il codice della Lezione 11 ( `MavenDate` ):

- `src/main/java/it/oop/core/Language.java`
- `src/main/java/it/oop/core/Date.java`
- `src/main/java/it/oop/core/ItalianDate.java`
- `src/main/java/it/oop/core/AmericanDate.java`
- `src/main/java/it/oop/core/Birthday.java`
- `src/main/java/it/oop/ui/Date.java`
- `src/main/java/it/oop/ui/MainDate.java`
- `src/test/java/it/oop/core/TestItalianDate.java`

Struttura dei file della lezione (path dalla cartella radice):



Argomenti trattati:

- Polimorfismo per ereditarietà: superclasse `Date` e sottoclassi `ItalianDate` e `AmericanDate`
- Sovrascrittura dei metodi ( `@Override` di `prettyPrint()` , `printFormat()` , `toString()` )
- Implementazione robusta del metodo `equals(Object other)` : verifica di identità ( `this == other` ), non nullità, controllo tipo con `instanceof` e casting
- Test di regressione con asserzioni su sottoclassi specializzate

## 1. Enum `Language`

```
In [1]: enum Language { IT, US }
```

## 2. Superclasse `Date` con metodo `equals` polimorfico

```
In [2]: class Date {
        private int day;
        private int month;
```

```

private int year;
static final String FORMAT_IT = "dd/mm/yyyy";
static final String FORMAT_US = "mm/dd/yyyy";
private Language lang;
private static final String[] MONTHS_IT = { "gennaio", "febbraio", "marzo", "aprile", "maggio",
"giugno", "luglio", "agosto", "settembre", "ottobre", "novembre", "dicembre" };
private static final String[] MONTHS_US = { "January", "February", "March", "April", "May", "June",
"July", "August", "September", "October", "November", "December"};

public Date(int day, int month, int year) {
    this.day = day;
    this.month = month;
    this.year = year;
    verify();
    lang = Language.IT;
}
public Date(int day, int month) {
    this(day, month, 2025);
}
public Date(Date other) {
    this.day = other.day;
    this.month = other.month;
    this.year = other.year;
    verify();
    lang = other.lang;
}

void verify() {
    if (year < 0 || month < 1 || month > 12)
        System.out.println("Illegal date!"); // Illegal date!
    else
        if (day < 1 || day > daysPerMonth(month))
            System.out.println("Illegal date!"); // Illegal date!
}

public int getDay() { return day; }
public int getMonth() { return month; }
public int getYear() { return year; }
public Language getlang() { return lang; }

public void setDay(int day) {
    this.day = day;
    verify();
}
public void setMonth(int month) {
    this.month = month;
    verify();
}
public void setYear(int year) {
    this.year = year;
    verify();
}

public static int daysPerMonth(int month) {
    int days;
    switch(month) {
        case 4:
        case 6:
        case 9:
        case 11:
            days = 30;
            break;
        case 2:
            days = 28;
            break;
        default:
            days = 31;
            break;
    }
    return days;
}

@Override
public boolean equals(Object other) {
    if (other == null) return false;

```

```

        if (this == other) return true;
        if (!(other instanceof Date)) return false;
        Date otherAsDate = (Date) other;
        return this.day == otherAsDate.getDay() &&
               this.month == otherAsDate.getMonth() &&
               this.year == otherAsDate.getYear();
    }
}

```

### 3. Sottoclasse `ItalianDate`

```

In [3]: class ItalianDate extends Date {
        private static final String[] MONTHS_IT = { "gennaio", "febbraio", "marzo", "aprile", "maggio",
        "giugno", "luglio", "agosto", "settembre", "ottobre", "novembre", "dicembre" };

        public ItalianDate(int day, int month, int year) {
            super(day, month, year);
        }

        public String printFormat() {
            return "dd/mm/yyyy";
        }

        public String prettyPrint() {
            return getDay() + " " + MONTHS_IT[getMonth()-1] + " " + getYear();
        }

        @Override
        public String toString() {
            return getDay() + "/" + getMonth() + "/" + getYear();
        }
    }

```

### 4. Sottoclasse `AmericanDate`

```

In [4]: class AmericanDate extends Date {
        private static final String[] MONTHS_US = { "January", "February", "March", "April", "May", "June",
        "July", "August", "September", "October", "November", "December" };

        public AmericanDate(int day, int month, int year) {
            super(day, month, year);
        }

        public String printFormat() {
            return "mm/dd/yyyy";
        }

        public String prettyPrint() {
            return MONTHS_US[getMonth()-1] + " " + getDay() + ", " + getYear();
        }

        @Override
        public String toString() {
            return getMonth() + "/" + getDay() + "/" + getYear();
        }
    }

```

### 5. Sottoclasse `BirthDay`

```

In [5]: class BirthDay extends Date {
        private static BirthDay instance = null;

        private BirthDay() {
            super(1, 1, 1970);
        }

        public static BirthDay getInstance() {
            if (instance == null)
                instance = new BirthDay();
        }
    }

```

```

        return instance;
    }
}

```

## 6. Classe `TestItalianDate` ed Esecuzione

```

In [6]: class TestItalianDate {
        private static ItalianDate date = new ItalianDate(1, 1, 1970);

        private static void printFormatTest() {
            assert date.printFormat().equals("dd/mm/yyyy");
        }

        public static void main(String[] args) {
            printFormatTest();
        }
    }
    TestItalianDate.main(null);

```

## 7. Classe `MainDate` ed Esecuzione

```

In [7]: class MainDate {
        public static void main(String[] args) {
            Date date = new Date(3, 11, 2025);
            ItalianDate itDate = new ItalianDate(3, 11, 2025);
            AmericanDate usDate = new AmericanDate(3, 11, 2025);
            System.out.println("date: " + date.toString()); // date: it.oop.core.Date@251a69d7
            System.out.println("itDate: " + itDate.toString()); // itDate: 3/11/2025
            System.out.println("usDate: " + usDate.toString()); // usDate: 11/3/2025
            System.out.println("itDate format: " + itDate.printFormat()); // itDate format: dd/mm/yyyy
            System.out.println("usDate format: " + usDate.printFormat()); // usDate format: mm/dd/yyyy
            System.out.println("date != itDate: " + date.equals(itDate)); // date != itDate: true
            System.out.println("itDate != usDate: " + itDate.equals(usDate)); // itDate != usDate: true
            System.out.println("date != 4/11/2025: " + date.equals(new ItalianDate(4, 11, 2025))); // date !=
4/11/2025: false
            Date id = new ItalianDate(3, 11, 2025);
            System.out.println(((ItalianDate) id).printFormat()); // dd/mm/yyyy
        }
    }
    MainDate.main(null);

```

```

date: REPL.$JShell$3$Date@2a5accd4
itDate: 3/11/2025
usDate: 11/3/2025
itDate format: dd/mm/yyyy
usDate format: mm/dd/yyyy
date != itDate: true
itDate != usDate: true
date != 4/11/2025: false
dd/mm/yyyy

```